



INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

REPORT

Title: Web Architecture Analysis and XSS Vulnerability Identification

Student Name: Ahmad Zubairu Garba

Student ID: 2025/FWSD/11463

Course: Web Application Security Essentials

Course Code: BVWS102

Instructor: AHMED BUKAR

9/NOV/2025

EXECUTIVE SUMMARY

This report presents findings from a hands-on lab exercise focused on identifying and mitigating cross-site scripting (XSS) vulnerabilities in web applications. Two sets of web pages were developed: vulnerable versions that allowed malicious scripts to execute, and secure counterparts that neutralized these threats through input sanitization. The exercise highlighted the risks of insecure coding practices, demonstrated effective countermeasures, and provided insights into future security improvements.

KEY DIFFERENCES BETWEEN VULNERABLE AND SECURE CODE

The primary distinction between the vulnerable and secure implementations lies in how user input is processed and displayed:

Vulnerable Code:

- Relied on `innerHTML` to insert user input directly into the page.
- Allowed malicious payloads such as `<script>alert('XSS')</script>` or `` to execute.
- Demonstrated that trusting user input without validation exposes applications to XSS attacks.

Secure Code:

- Implemented an `escapeHTML` function to sanitize input.
- Converted special characters (`<`, `>`, `&`, `"`, `'`) into harmless HTML entities.
- Ensured that payloads were displayed as plain text rather than executed as code.
- Reinforced the principle that **user input must always be treated as untrusted data**.

This difference underscores the importance of secure coding practices. Vulnerable code interprets input as instructions, while secure code ensures input is treated strictly as data.

MOST SURPRISING FINDING

The most unexpected outcome of the lab was the variety of attack vectors beyond the `<script>` tag. While script injection was anticipated, other payloads such as broken image tags with `onerror` attributes and SVG elements with `onload` handlers also

executed successfully in the vulnerable pages. This demonstrated that attackers can exploit multiple HTML elements and event attributes to achieve the same malicious outcome.

The lesson learned is that XSS is not limited to obvious script tags; any input that can be interpreted as executable code poses a risk. This finding emphasizes the need for comprehensive sanitization and a defense-in-depth approach.

RECOMMENDATION FOR FUTURE PROJECTS

Based on the lab results, the following recommendations are proposed for future web development projects:

1.Implement Output Encoding:

Always escape user input before rendering it in the browser. This ensures that potentially malicious characters are displayed as text rather than executed.

2.Adopt Content Security Policy (CSP):

Configure CSP headers to restrict script execution sources and block inline scripts. This adds an additional layer of protection against XSS.

3.Avoid Inline Event Handlers:

Replace constructs such as `` onclick="..."`` with safer alternatives like `` addEventListener`` in JavaScript. This reduces exposure to injection attacks.

4.Validate Input on Both Client and Server:

Client-side validation improves user experience, but server-side validation is essential to enforce security. Never rely solely on client checks.

5.Conduct Regular Security Testing:

Incorporate vulnerability scanning and penetration testing into the development lifecycle to identify and remediate issues early.

CONCLUSION

The lab demonstrated that insecure coding practices, such as directly inserting user input into the DOM, create significant XSS vulnerabilities. Secure coding practices, including input sanitization and output encoding, effectively neutralize these threats.

The most surprising finding was the breadth of possible attack vectors, proving that attackers exploit more than just script tags.

Moving forward, the adoption of output encoding, CSP, and robust validation will be critical to ensuring secure applications. By embedding these practices into the development process, organizations can reduce risk, protect user data, and maintain trust in their web platforms.